Mochilas por fuerza bruta

Práctica breve con respuestas razonadas

ALGOED · Material de apoyo para la clase de 60 minutos
Trabaja en C++ con arreglos, ciclos, residuo y división entera. Resuelve primero; consulta las respuestas al final. Los dos programas completos y sus casos de entrada están en `Codigo C++`.

1. Selección binaria · 10 minutos

Hay cuatro artículos de pesos **1, 2, 10 y 4**. Cada artículo puede elegirse como máximo una vez. No hay ganancias distintas del peso. Busca el mayor peso total que no exceda la capacidad.

1. Con capacidad **15**, escribe una selección óptima y su suma.
2. Repite con capacidad **9**. Explica por qué dejar espacio libre puede ser correcto.
3. ¿Cuántas selecciones debes considerar? ¿Está incluida la selección vacía?
4. En esta entrega, el primer artículo es el dígito de la izquierda. ¿Qué selección genera el código decimal **11**? ¿Y el código **1**?
5. Completa el algoritmo. ¿Por qué hay que copiar la selección cuando mejora la respuesta?

```
for (int codigo = 0; codigo < total; codigo++) {
    cargarCromosoma(codigo, n, seleccion);
    long long peso = 0;
    // Suma solo los articulos seleccionados.
    if (/* cabe y mejora */) {
        // Conserva el peso y la seleccion.
    }
}
```

Contraejemplo: capacidad 6, pesos 4,3,3. ¿Qué devuelve “llevar primero el más pesado que quepa”? ¿Cuál es la selección óptima? Los dos artículos de peso 3 son distintos.

2. Varias mochilas · 8 minutos

Dispones de dos mochilas, M1 con capacidad **5** y M2 con capacidad **4**, y de tres artículos A,B,C con pesos **4,3,2**. Cada artículo puede ir a una mochila o no usarse. Aquí el objetivo es maximizar el peso total utilizado sin exceder ninguna capacidad.

1. Define qué significan los dígitos 0,1,2 de una asignación.
2. ¿Cuántas asignaciones completas existen?
3. Interpreta `2 1 1` en el orden A,B,C. Comprueba las dos capacidades.
4. Convierte ese cromosoma a decimal, usando el primer artículo como dígito más significativo.

5. Explica por qué resolver cada mochila por separado puede reutilizar un artículo y producir una respuesta inválida.

Extensión para practicar: adapta el contador binario a base 3. Para cada candidato reinicia `carga[2]`, suma el peso en su destino si no es cero, rechaza si una carga excede su capacidad y conserva la mayor suma de ambas. La asignación `2 1 1` ya permite comprobar que el mejor total es 9: usa todo el peso disponible.

3. Po · LAB1 2024-1, pregunta 1 · 15 minutos

Encuentra **cualquier asignación válida**. No hay capacidad de peso ni se exige maximizar poder. Cada arma se usa a lo sumo una vez; puede quedar sin usar. Los requisitos de un arma deben estar **en su misma mochila**. El poder total debe ser **estrictamente mayor** al del guerrero.

Guerrero	Poder a superar	Tipos permitidos
1	120	2
2	160	1, 3
3	80	3

Arma	Poder	Tipo	Requisitos
Z	60	3	—
P	80	1	Z
R	38	2	—
D	25	2	R
E	49	2	—
F	57	1	—
G	68	3	—
H	35	2	Z, E
I	62	2	R
J	42	2	—
K	36	1	Z
L	54	3	—

1. Define el cromosoma y la base. ¿Cuántas asignaciones hay con 12 armas y 3 guerreros?
2. Verifica la propuesta del enunciado: M1={R,D,I}, M2={Z,P,F}, M3={G,L}. Revisa suma, tipo, requisito y exclusividad.
3. ¿Basta con que Z esté seleccionada en alguna mochila para que P sea válida?

4. Demuestra por qué H no puede usarse con estos guerreros. ¿Sería correcto prohibir la letra H dentro de un algoritmo general?
5. Ejecuta `po.cpp`. Si imprime M1={R,I,J}, M2={Z,P,K}, M3={G,L}, ¿es incorrecto por ser diferente al ejemplo? Justifica.
6. Explica por qué el buscador puede devolver la primera asignación válida. ¿Qué cambiaría si pidieran maximizar el poder total?

4. Detecta el fallo · 5 minutos

Decide si existe solución y señala la condición que lo determina:

Caso	Datos
A	Un guerrero de poder 10 admite tipo 1. Única arma: poder 10, tipo 1, sin requisitos.
B	Un guerrero de poder 10 admite tipo 1. A: poder 6, tipo 1, requiere B. B: poder 5, tipo 1, sin requisitos.
C	Un guerrero de poder 10 admite tipo 1. Única arma: poder 20, tipo 2, sin requisitos.
D	Dos guerreros de poder 5 admiten tipo 1. Solo hay una arma de poder 6, tipo 1, sin requisitos.

Respuestas y comprobaciones

1. Mochila binaria

- Capacidad 15: selección `1 0 1 1`; 1+10+4=15. Es óptima porque alcanza la capacidad, que es una cota superior.
- Capacidad 9: selección `1 1 0 1`; 1+2+4=7. El artículo de peso 10 no cabe y todos los demás juntos suman 7; no puede conseguirse 8 ni 9.
- Hay 2^4=16 selecciones, incluida `0 0 0 0`.
- Código 11: `1 0 1 1`; 11=8+2+1. Código 1: `0 0 0 1`, que elige el artículo de peso 4. Esta convención también se usa en `po.cpp`.
- La condición es `peso <= capacidad && peso > mejorPeso`. Copia la selección porque el arreglo candidato cambiará en la siguiente iteración. El programa conserva la primera selección que alcanza cada mejor peso.
- Con capacidad 6 y pesos 4,3,3: el criterio voraz elige 4 y deja 2 libres; la selección óptima es `0 1 1`, suma 6. Factible no significa óptimo.

2. Dos mochilas

Los destinos son 0=sin usar, 1=M1 y 2=M2. Hay 3^3=27 asignaciones. `2 1 1` coloca A en M2 y B,C en M1: M2 pesa 4≤4 y M1 pesa 3+2=5≤5. El total 9 es óptimo porque utiliza los tres artículos. Su código decimal es 2·9+1·3+1=22. Un único destino por artículo impide que A se repita en ambas mochilas.

3. Po

Un dígito por arma en el orden Z,P,R,D,E,F,G,H,I,J,K,L. Base 4: 0=sin usar y 1,2,3=mochila correspondiente. Hay 4^{12}=16 777 216 candidatos.

Solución del enunciado	Poder	Comprobación de requisitos
M1: R,D,I	38+25+62=125 > 120	D e I requieren R, que está allí. Todos son tipo 2.
M2: Z,P,F	60+80+57=197 > 160	P requiere Z, que está allí. Tipos 1 y 3 permitidos.
M3: G,L	68+54=122 > 80	No requieren otras armas. Ambos son tipo 3.

No hay armas repetidas. E,H,J,K quedan sin usar, lo cual está permitido. El cromosoma de esa solución es `2 2 1 1 0 2 3 0 1 0 0 3`.

Z debe tener exactamente el mismo destino que P si P se usa. H es tipo 2 y necesita Z de tipo 3; ningún guerrero admite ambos tipos, de modo que H no puede participar en una solución de estos datos. No se debe prohibir H por nombre: otras entradas podrían tener otros tipos permitidos.

La primera solución del programa también es válida: M1 R,I,J suma 142; M2 Z,P,K suma 176; M3 G,L suma 122. I tiene a R, y P y K tienen a Z en su mochila. Su cromosoma es `2 2 1 0 0 0 3 0 1 1 2 3`. Con el orden de esta implementación aparece tras 10 751 068 candidatos probados. El orden de enumeración puede cambiar qué solución se encuentra primero.

El corte en la primera válida es correcto porque se pide existencia. Para maximizar, habría que comparar las soluciones factibles y conservar la mejor, salvo que se demuestre que se alcanzó una cota superior. La enumeración sigue siendo exponencial; los límites pequeños del laboratorio son esenciales.

4. Casos mínimos

A: no; 10 no supera estrictamente 10. **B:** sí; A y B juntas suman 11 y satisfacen el requisito. **C:** no; el tipo 2 no está admitido. **D:** no; una única arma no puede utilizarse en dos mochilas.

Comprobación antes de entregar tu solución

Debes poder explicar el significado de cada dígito, justificar el total de candidatos y demostrar que todas las restricciones se comprueban. Prueba una entrada con solución y otra sin solución, un empate de poder, un requisito ausente y un tipo incompatible. El archivo `Codigo C++/LEEME.md` contiene los comandos y los resultados de los casos incluidos.

Procedencia: tablas y reglas de Po tomadas del enunciado PUCP LAB1 2024-1, pregunta 1, incluido en esta entrega. La mochila inicial adapta el ejemplo del banco local de ALGOED. Los ejercicios de transferencia y estas explicaciones son material de apoyo; no son una solución oficial de la universidad.